

Deep Learning & NLP Exam Prep Guide

Comprehensive Quick-Reference: Architectures, Language Modeling, Prompting, Attention, Alignment & Numericals

1. Core Architectures

Transformers (Self-Attention, Encoder-Decoder, Autoregressive vs Bidirectional)

An architecture processing sequences in parallel using self-attention to map global dependencies. Encoders capture full bidirectional context simultaneously, while autoregressive decoders predict tokens sequentially by masking future positions.

Numerical Example: Autoregressive vs Bidirectional Masking

Given an input sequence of 3 tokens. A bidirectional encoder uses a full attention matrix where all cells are active (all 9 elements available). An autoregressive decoder applies a lower-triangular causal mask: Row 1 sees 1 token, Row 2 sees 2, Row 3 sees 3, restricting active attention connections to $(3 \times 4) / 2 = 6$ positions.

Benefits	Limitations
- Parallel training avoids sequential bottlenecks of RNNs. - Captures long-range dependencies effectively across tokens.	- Quadratic compute/memory scaling $O(N^2)$ with sequence length. - Lacks inductive bias for sequential order, requiring positional encodings.

Seq2Seq Models (Encoder-Decoder with Attention)

An architecture mapping variable-length input sequences to variable-length outputs using an encoder to process source sequences and a decoder to generate targets. Dynamic cross-attention allows the decoder to focus selectively on specific encoder hidden states during generation.

Toy Example: Machine Translation

Source: "Je t'aime" → Encoder Hidden States $[h_1, h_2]$. When the decoder generates target word "I", its cross-attention mechanism computes alignment scores matching "I" to h_1 ("Je") with high probability ($\alpha = 0.95$).

Benefits	Limitations
- Flawlessly processes distinct input/output lengths. - Attention eliminates the information bottleneck of fixed-size vectors.	- Autoregressive decoding is slow due to token-by-token generation. - Prone to exposure bias during standard teacher-forcing training.

BERT (Masked Language Modeling, Bidirectional Context)

An encoder-only Transformer trained using a Masked Language Model (MLM) objective to predict hidden tokens by looking both left and right. This allows the model to learn deep, non-directional contextual representations for downstream tasks.

Numerical Example: MLM Masking Setup

For a sequence of 100 tokens, BERT randomly selects 15% (15 tokens) for prediction. Out of these 15 selected tokens, 80% (12 tokens) are replaced with the **[MASK]** token, 10% (1.5 $\rightarrow$ 2 tokens) are replaced with a random word, and 10% (1.5 $\rightarrow$ 1 token) are left completely unchanged.

Benefits

- Rich, bidirectional context representations for language understanding.
- Highly adaptable via fine-tuning for classification and QA.

Limitations

- Mismatch artifact because **[MASK]** tokens never appear during downstream fine-tuning.
- Cannot perform standard autoregressive natural text generation efficiently.

GPT (Causal LM, Autoregressive Generation)

A decoder-only Transformer pre-trained on a causal language modeling objective to predict the next token given preceding tokens. It utilizes causal masking to block information leakage from future tokens, making it optimal for generation.

Numerical Example: Causal Matrix Masking

For a sequence of length 3, the raw attention energy matrix **S** is modified by adding a causal mask **M** before Softmax. Elements where $j > i$ are set to $-\infty$. Row 2 retains actual scores for indices 1 and 2, but index 3 becomes $e^{-\infty} = 0$ after computing Softmax.

Benefits

- Superb, highly coherent open-ended natural text generation capabilities.
- Exhibits powerful zero/few-shot in-context learning without parameter updates.

Limitations

- Strict left-to-right processing misses future contextual cues.
- Highly susceptible to error propagation and structural hallucination.

BART (Hybrid Encoder-Decoder)

A sequence-to-sequence model combining a bidirectional encoder and an autoregressive decoder, trained by corrupting text with arbitrary noise and learning to reconstruct the original clean text.

Toy Example: Text Infilling Objective

Original: "The cat sat on the mat". Corrupted input to Encoder: "The cat [MASK] mat" (token deletion and infilling). The Decoder is trained step-by-step to predict and output the original complete string: "The cat sat on the mat".

Benefits

- Combines bidirectional flexibility with robust generation.
- Top-tier performance on text summarization and translation.

Limitations

- Double-component structure increases parameter count and training compute.
- Complex pre-training architecture compared to plain decoder-only setups.

CNNs for Segmentation (Encoder-Decoder / U-Net)

An architecture using a contracting path (encoder) to extract downsampled high-level features, paired with an expanding path (decoder) to upsample maps. Skip connections copy high-resolution features from encoder to decoder to preserve precise structural localization.

Numerical Example: Spatial Dimension Resolution Tracking

Input image size is **256 *imes* 256**. After 3 layers of Max-Pooling (each scaling down by factor of 2), feature size drops to **32 *imes* 32**. The decoder uses Transposed Convolutions to scale back up to **256 *imes* 256**, merging features via skip connections at each corresponding stage.

Benefits

- Skip connections precisely restore pixel-level spatial boundaries.
- Parameter efficiency via translation-invariant weight sharing.

Limitations

- Restricted receptive fields make capturing long-range context difficult.
- High structural memory overhead from storing raw intermediate activations.

RNNs, LSTMs, GRUs (Gating Mechanisms & Applications)

Sequential networks maintaining recurrent internal states. LSTMs and GRUs introduce specialized mathematical gating mechanisms to control information flow, preventing standard gradients from vanishing across long historical horizons.

Numerical Example: LSTM Forget Gate Operation

An LSTM forget gate activation evaluates to $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) = 0.05$. The previous long-term cell memory state value was $C_{t-1} = 12.0$. The network scales this memory to $f_t \text{ *imes* } C_{t-1} = 0.05 \text{ *imes* } 12.0 = 0.60$, effectively clearing 95% of past historical info.

Benefits

- Gating mechanisms explicitly mitigate the vanishing gradient problem.
- Naturally handles sequential inputs of dynamically changing lengths.

Limitations

- Sequential step execution prevents massive parallelization across tokens.
- Hardware scaling bottlenecks restrict training efficiency on huge datasets.

2. Language Modeling Concepts

N-gram Models & Smoothing Techniques

Statistical text models predicting next tokens based on fixed-length context windows of size $N-1$. Smoothing reallocates probability mass from frequent observations to unseen combinations to avoid dividing by zero.

Numerical Example: Laplace (+1) Smoothing

Vocabulary size $V = 1000$. Unigram count of word "cat" is $C(\text{"cat"}) = 10$. Total corpus size $N = 50000$. Unsmoothed probability $P = 10/50000 = 0.00020$. Smoothed probability $P_{\text{Laplace}} = (10+1)/(50000+1000) = 11/51000 \text{ *approx* } 0.000216$.

Benefits

- Computationally lightweight; simple frequentist counting.
- Kneser-Ney smoothing handles unseen contexts elegantly.

Limitations

- Severe data sparsity issues as context length N increases.
- Fails to capture any semantic similarities between related words.

Perplexity (Definition, Calculation, Relation to Cross-Entropy)

An information-theoretic metric quantifying language model uncertainty, calculated as the exponentiated cross-entropy loss. Lower perplexity reflects a highly confident model with a narrower, more accurate branching factor.

Requested Example: "i love cat" Perplexity

Given words: "i", "love", "cat". Assumed model conditional probabilities: $P(\text{ext}\{"i"\}) = 0.4$, $P(\text{ext}\{"love"\}|\text{ext}\{"i"\}) = 0.5$, and $P(\text{ext}\{"cat"\}|\text{ext}\{"i love"\}) = 0.1$.

Sequence Joint Probability: $\text{prod } P = 0.4 \times 0.5 \times 0.1 = 0.02$.

Sequence Length $W = 3$.

Perplexity Score Calculation: $PP = (0.02)^{-1/3} = \sqrt[3]{50} \approx 3.684$.

Benefits • Directly measures absolute probabilistic modeling text quality. • Mathematically linked to Cross-Entropy via equation: $PP = 2^H$.	Limitations • Extremely sensitive to vocabulary text filtering and tokenization variations. • Low perplexity does not inherently guarantee factual correctness or logical reasoning.
---	---

Evaluation Metrics (Precision, Recall, F1, Perplexity)

Quantitative criteria used to assess models: Precision tracks exact correctness, Recall measures target coverage, F1 computes their balanced harmonic mean for classification, and Perplexity evaluates distribution fits for language generation.

Numerical Example: Precision, Recall, and F1 Score Calculation

A model predicts 10 tokens as entities: 8 are correct (True Positives, $TP=8$), 2 are wrong (False Positives, $FP=2$). The true text label data contains 12 total actual entities, meaning 4 were completely missed (False Negatives, $FN=4$).

$\text{ext}\{Precision\} = 8/(8+2) = 0.80$; $\text{ext}\{Recall\} = 8/(8+4) = 0.667$.

$\text{ext}\{F1\ Score\} = 2 \times \text{rac}\{0.80 \times 0.667\}\{0.80 + 0.667\} = \text{rac}\{1.067\}\{1.467\} \approx 0.727$.

Benefits • F1 provides a balanced assessment on highly imbalanced class distributions. • Standardized benchmarks across machine learning applications.	Limitations • Classification metrics fail to capture nuances of free-form generation. • Vulnerable to target distribution shifts and edge-case anomalies.
---	--

3. Prompting Techniques

Zero-shot vs Few-shot Prompting

Zero-shot queries a language model directly without providing any input-output examples. Few-shot includes a small set of structured input-output pairs within the prompt context to demonstrate the intended task format and constraints.

Toy Example: Sentiment Analysis Classification

Zero-shot: "Review: 'Great food!' Sentiment:" → Model outputs "Positive".

Few-shot: "Review: 'Bad service' → Negative; Review: 'Loved it' → Positive; Review: 'Delicious meal' →" → Model extracts pattern and outputs "Positive".

Benefits • Requires zero parameter fine-tuning or weight modification. • Few-shot clarifies complex formatting rules and edge-case behaviors.	Limitations • Zero-shot struggles with complex, structurally specific instructions. • Few-shot consumes valuable context window space and increases inference costs.
---	--

Chain-of-Thought Prompting

A technique instructing the model to generate intermediate step-by-step reasoning paths before producing the final answer. This activates structured computation paths for complex multi-step reasoning tasks.

Toy Example: Arithmetic Logic Tracking

Prompt: "John has 5 apples. He buys 2 more bags containing 3 apples each. Let's think step by step."

Model output: "1. John starts with 5 apples. 2. 2 bags of 3 apples equals **2 times 3 = 6** apples. 3. Total is **5 + 6 = 11**.

Answer: 11."

Benefits • Significantly boosts logical accuracy on math and reasoning tasks. • Provides an interpretable, inspectable trace of the model's logic.	Limitations • Increases output token count, generation latency, and inference costs. • If an early reasoning step hallucinates, errors propagate to the final answer.
--	---

Instruction, Role, and Structured/Deliberate Prompting

Advanced prompting paradigms framing tasks via clear execution commands, persona specifications ("Act as an expert"), and explicit markdown formatting constraints to govern model behavior and output structure.

Toy Example: Role + Structured Combination

Prompt: "[Role]: Act as a Senior DevOps Engineer. [Instruction]: Summarize the server crash log down to a 1-sentence takeaway. Return the output in a clean JSON block: {'takeaway': '...'}"

Benefits • Personas adjust tone, vocabulary, and depth to match target domains. • Structured constraints allow direct parsing by downstream application code.	Limitations • Models can exhibit instruction drift or forget constraints in long contexts. • Prone to prompt injection and jailbreaking vulnerabilities.
---	--

4. Attention Mechanisms

Self-Attention vs Cross-Attention

Self-attention computes alignment scores within a single sequence to capture internal relationships. Cross-attention connects separate sequences by deriving Queries (**Q**) from the target decoder and Keys (**K**) and Values (**V**) from the source encoder.

Numerical Example: Dimensional Input Sourcing

Decoder hidden dimension state provides Query vector $q \in \mathbb{R}^d$. Encoder sequence provides Keys matrix $K \in \mathbb{R}^{L \times d}$. The attention weights are calculated via matrix-vector multiplication: $\text{Weights} = \text{Softmax}(q K^T / \sqrt{d})$, yielding an L -dimensional probability distribution over encoder tokens.

Benefits

- Cross-attention dynamically aligns multimodal or bilingual sequences.
- Self-attention enables parallel processing across token sequences.

Limitations

- Both incur heavy compute and memory overheads during scaling.
- Requires deep architectures to extract complex multi-layered relationships.

Global vs Local Attention & Softmax Normalization

Global attention computes weights across all tokens in a sequence, while local attention limits this calculation to a fixed window. Softmax converts raw attention energies into a valid probability distribution summing to 1.

Numerical Example: Softmax Normalization Scaled Vector

Raw attention dot-products for a token with 2 neighbors are $S = [4.0, 2.0]$. Scale factor $\sqrt{d_k} = 2.0$. Scaled vector $S' = [2.0, 1.0]$.

Exponentials: $e^2 \approx 7.389$, $e^1 \approx 2.718$. Sum = 10.107.

Normalized attention weights: $\text{Alpha} = [7.389/10.107, 2.718/10.107] = [0.731, 0.269]$.

Benefits

- Local attention reduces time complexity to linear $O(N \times W)$.
- Softmax ensures gradients flow smoothly during backpropagation.

Limitations

- Local attention cannot capture dependencies that fall outside its window.
- Softmax can suffer from numeric instability and vanishing gradients if raw scores are too large.

5. Training & Alignment

RLHF (Reward Modeling from Preferences) & Instruction Fine-Tuning (IFT)

IFT updates base models on supervised instruction-response pairs to teach task compliance. RLHF takes this further by training a Reward Model on human preference rankings, using Reinforcement Learning (PPO) to align the model with safety and helpfulness.

Toy Example: Preference Optimization Alignment

Prompt: "Write a joke about safety." Output A (funny but risky), Output B (safe and helpful). Human annotators rank $B > A$. The Reward Model learns to score $R(\text{Output B}) = +1.5$ and $R(\text{Output A}) = -1.2$, guiding the policy network toward safer text generation.

Benefits

- Aligns model behavior with human values, safety guidelines, and intent.
- IFT bridges the gap between next-token prediction and chat utility.

Limitations

- RLHF training can be unstable and sensitive to hyperparameters.
- Risk of 'reward hacking,' where the model generates high-scoring but low-quality text.

6. Numerical Practice Topics

Numerical Workout 1: Multi-Token Perplexity Verification

Given a 4-token sentence evaluation path sequence: $S = [w_1, w_2, w_3, w_4]$.

Extracted conditional model generation probabilities: $P(w_1)=0.50$, $P(w_2|w_1)=0.40$, $P(w_3|w_1 w_2)=0.25$, $P(w_4|w_1 w_2 w_3)=0.10$.

Step 1: Compute Joint Probability

$$P(S) = 0.50 \times 0.40 \times 0.25 \times 0.10 = 0.0050.$$

Step 2: Calculate Geometric Inverse Average (Perplexity)

Sentence Length $N = 4$.

$$PP(S) = [P(S)]^{\{-\frac{1}{N}\}} = (0.0050)^{\{-0.25\}} = \left(\frac{1}{200}\right)^{\{-0.25\}} = (200)^{\{0.25\}} \approx 3.7606.$$

Numerical Workout 2: N-gram Bigram Kneser-Ney / Absolute Discounting

Corpus data: Bigram count $C(\text{"deep learning"}) = 8$. Unigram count $C(\text{"deep"}) = 20$. Vocabulary $V=500$.

Absolute Discount Parameter factor constant set at: $d = 0.75$.

Unique words continuing after context "deep": $N_{+1}(\text{"deep"} \text{ ullet}) = 4$.

Normalized global continuation probability for target word "learning": $P_{\{\text{continuation}\}}(\text{"learning"}) = 0.02$.

Smoothed Equation Formulation:

$$P_{\{\text{KN}\}} = \frac{\max(C(\text{"deep learning"}) - d, 0)}{C(\text{"deep"}) + \frac{d \cdot N_{+1}(\text{"deep"} \text{ ullet})}{C(\text{"deep"})}} \times P_{\{\text{continuation}\}}(\text{"learning"})$$

Step-by-Step Value Substitution Execution:

$$\text{Term 1 (Discounted Base)} = \frac{8 - 0.75}{20} = \frac{7.25}{20} = 0.3625.$$

$$\text{Term 2 Lambda Scale weight coefficient} = \frac{0.75 \times 4}{20} = \frac{3.0}{20} = 0.150.$$

$$\text{Combined Final Metric Result: } P_{\{\text{KN}\}} = 0.3625 + (0.150 \times 0.02) = 0.3625 + 0.0030 = 0.3655.$$

Numerical Workout 3: LSTM Gate Activation Progression

Hidden state vector $h_{t-1} = [0.5, -0.2]$, Input vector $x_t = [1.0, 0.0]$. Weight matrix row slice $W_i = [0.8, 0.4, 0.5, -0.1]$, bias $b_i = -0.1$.

Step 1: Compute Linear Combination

$$\text{Concatenated vector } v = [h_{t-1}, x_t] = [0.5, -0.2, 1.0, 0.0].$$

$$\text{Net input } z = (0.5 \times 0.8) + (-0.2 \times 0.4) + (1.0 \times 0.5) + (0.0 \times -0.1) + (-0.1)$$

$$z = 0.40 - 0.08 + 0.50 + 0.0 - 0.10 = 0.72.$$

Step 2: Apply Sigmoid Transfer Function Activation

$$\text{Input gate activation value: } i_t = \text{sigma}(0.72) = \frac{1}{1 + e^{-0.72}} = \frac{1}{1 + 0.4868} \approx 0.6725.$$

Numerical Workout 4: Attention Weight Vector Computation

Query vector $q = [2.0, 0.0, 2.0]$. Key matrix matrix transpose $K^T = \begin{bmatrix} 2.0 & 1.0 & 0.0 \\ 0.0 & 2.0 & 2.0 \\ 2.0 & 0.0 & 0.0 \end{bmatrix}$. Dimension size $d_k = 3$.

Step 1: Vector-Matrix Inner Scaling Products

$$S_1 = q \cdot K_1 = (2 \times 2) + (0 \times 0) + (2 \times 2) = 4 + 0 + 4 = 8.0.$$

$$S_2 = q \cdot K_2 = (2 \times 1) + (0 \times 2) + (2 \times 0) = 2 + 0 + 0 = 2.0.$$

Step 2: Division by Square Root Scaling Dimension Factor

Let scale factor be approximated directly as $\sqrt{3} \approx 1.732$.

$$S'_1 = 8.0 / 1.732 \approx 4.619; \quad S'_2 = 2.0 / 1.732 \approx 1.155.$$

Step 3: Softmax Exponential Summation Normalization

$$e^{4.619} \approx 101.393, \quad e^{1.155} \approx 3.174. \quad \text{Sum total} = 104.567.$$

$$\alpha_1 = 101.393 / 104.567 \approx 0.9696; \quad \alpha_2 = 3.174 / 104.567 \approx 0.0304.$$

Final evaluated attention probability distribution layout vector: $\alpha = [0.9696, 0.0304]$.